

Documentation — Serveur Web sur Proxmox | elmzn.be

DOCUMENTATION

Déploiement d'un serveur web

sur infrastructure Proxmox + LXC + Nginx

Auteur : dexter_elmzn

Version : 1.0 — Mars 2026

Infrastructure : Proxmox VE + LXC Ubuntu 24.04 + Nginx

0. Prérequis & architecture

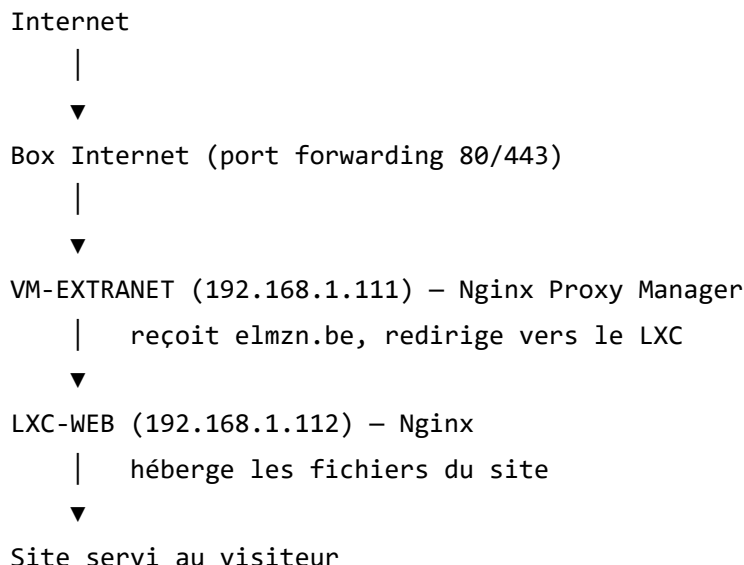
Avant de commencer, voici ce dont tu as besoin et comment les composants s'articulent.

0.1 Prérequis

- Un serveur Proxmox VE installé et accessible via son interface web (:8006)
- Un accès SSH root sur le host Proxmox
- Un nom de domaine (OVH ou autre) avec accès à la zone DNS
- Les ports 80 et 443 forwardés depuis ta box vers la VM/LXC exposée

0.2 Architecture cible

Le flux de trafic suit ce chemin :



0.3 Composants utilisés

Composant	Rôle	IP / Port
Proxmox VE	Hyperviseur — gère VMs et LXC	192.168.1.100:8006
VM-EXTRANET	DMZ — Nginx Proxy Manager (NPM)	192.168.1.111
LXC-WEB (ID 102)	Serveur web — Nginx + sites	192.168.1.112:80
NPM	Reverse proxy + SSL Let's Encrypt	:80 / :443 / :81 (dashboard)
OVH DNS	Résolution de domaine publique	elmzn.be → IP publique

1. Création du LXC Ubuntu

Un LXC (Linux Container) est un conteneur léger qui partage le kernel Proxmox. Il consomme 50 à 200 MB de RAM au repos, contre 800 MB pour une VM classique. C'est le choix idéal pour héberger des sites web.

1.1 Télécharger le template Ubuntu 24.04

Se connecter en SSH sur le host Proxmox, puis exécuter :

```
# Update template list
pveam update

# Verify Ubuntu 24.04 LTS is available
pveam available | grep ubuntu

# Download the template
pveam download local ubuntu-24.04-standard_24.04-2_amd64.tar.zst
```

i Pourquoi Ubuntu 24.04 LTS ?

LTS = Long Term Support : supportée jusqu'en 2029 (5 ans). Idéale pour un serveur qui tourne 24/7 sans réinstallation fréquente. Ne pas utiliser Ubuntu 25.04 (non-LTS, support expire en janvier 2026).

1.2 Créer et démarrer le LXC

Toujours depuis le shell Proxmox :

```
pct create 102 local:vztmp1/ubuntu-24.04-standard_24.04-2_amd64.tar.zst \
--hostname lxc-web \
--cores 2 \
--memory 512 \
--swap 512 \
--rootfs local-lvm:8 \
--net0 name=eth0,bridge=vmbr0,ip=192.168.1.112/24,gw=192.168.1.1 \
--nameserver 1.1.1.1 \
--unprivileged 1 \
--features nesting=1 \
--start 1
```

Paramètre	Valeur	Explication
--hostname	lxc-web	Nom du conteneur
--cores	2	Nombre de vCPUs alloués
--memory	512	RAM en MB (suffisant pour Nginx)
--rootfs	local-lvm:8	Disque de 8 GB sur le stockage local
--ip	192.168.1.112/24	IP fixe dans le LAN

Paramètre	Valeur	Explication
--unprivileged	1	Conteneur non-root (sécurité)
--features nesting	1	Permet Docker dans le LXC si besoin
--start	1	Démarre automatiquement après création

1.3 Activer le démarrage automatique au boot

Pour que le LXC redémarre automatiquement quand Proxmox redémarre :

```
# Enable autostart for the LXC
pct set 102 --onboot 1
```

```
# Verify
pct config 102 | grep onboot
# Expected output: onboot: 1
```

i Faire la même chose pour les VMs

```
qm set 100 --onboot 1 # VM-EXTRANET
qm set 101 --onboot 1 # VM-INTRANET
```

1.4 Entrer dans le LXC

```
# Enter LXC from Proxmox host
pct enter 102
```

```
# You should see this prompt:
# root@lxc-web:~#
```

2. Installation et configuration de Nginx

Nginx est le serveur web qui va servir les fichiers de tes sites. Toutes les commandes suivantes s'exécutent à l'intérieur du LXC (après `pct enter 102`).

2.1 Installer Nginx

```
# Update packages and install Nginx
apt update && apt upgrade -y && apt install -y nginx curl

# Verify Nginx is running
systemctl status nginx
# Expected: Active: active (running)
```

2.2 Créer la structure du site

```
# Create site directory
mkdir -p /var/www/elmzn.be

# Set correct permissions (www-data = Nginx user)
chown -R www-data:www-data /var/www/elmzn.be
chmod -R 755 /var/www/elmzn.be
```

Pour chaque nouveau site, créer un dossier dédié dans `/var/www/` :

```
# Pattern for additional sites
mkdir -p /var/www/blog.elmzn.be
mkdir -p /var/www/app.elmzn.be
chown -R www-data:www-data /var/www/
```

2.3 Créer le virtual host Nginx

Un virtual host permet à Nginx de servir plusieurs sites sur le même serveur en se basant sur le nom de domaine.

```
# Create the virtual host config
nano /etc/nginx/sites-available/elmzn.be
```

Contenu du fichier de configuration :

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name elmzn.be www.elmzn.be;
    root /var/www/elmzn.be;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    # Security headers
    add_header X-Frame-Options "SAMEORIGIN";
    add_header X-Content-Type-Options "nosniff";
    add_header X-XSS-Protection "1; mode=block";

    # Logs
    access_log /var/log/nginx/elmzn.be.access.log;
    error_log /var/log/nginx/elmzn.be.error.log;
}
```

i Note sur default_server

Le paramètre `default_server` sur la ligne `listen` permet à Nginx de servir ce site même quand aucun header `Host` n'est envoyé (ex: accès par IP directe). Un seul virtual host peut avoir `default_server` — généralement le site principal.

2.4 Activer le site

```
# Enable site (create symlink)
ln -s /etc/nginx/sites-available/elmzn.be /etc/nginx/sites-enabled/

# Disable default Nginx page
rm /etc/nginx/sites-enabled/default

# Test configuration syntax
nginx -t
# Expected: configuration file test is successful

# Reload Nginx to apply changes
systemctl reload nginx
```

2.5 Tester Nginx en local

```
# Test with Host header (simulates browser request)
curl -H "Host: elmzn.be" http://localhost

# Test by IP (works with default_server)
curl http://192.168.1.112
```

⚠ Si la page par défaut Nginx s'affiche encore

Vérifier qu'il n'y a pas de fichier `default` dans `/etc/nginx/sites-enabled/`

Commande : `ls /etc/nginx/sites-enabled/`

Si présent : `rm /etc/nginx/sites-enabled/default && systemctl reload nginx`

3. Déployer un site statique

Un site statique est composé de fichiers HTML, CSS et JS — aucun serveur applicatif requis. Nginx les sert directement.

3.1 Structure des fichiers recommandée

```
/var/www/elmzn.be/  
├─ index.html      ← page d'accueil  
├─ css/  
│   └─ style.css  
├─ js/  
│   └─ main.js  
└─ assets/  
    └─ images/
```

```
# Create folders  
mkdir -p /var/www/elmzn.be/css  
mkdir -p /var/www/elmzn.be/js  
mkdir -p /var/www/elmzn.be/assets/images
```

3.2 Lier CSS et JS dans index.html

```
<!DOCTYPE html>  
<html lang="fr">  
<head>  
  <meta charset="UTF-8">  
  <meta name="viewport" content="width=device-width, initial-scale=1.0">  
  <title>elmzn.be</title>  
  <!-- CSS in <head> -->  
  <link rel="stylesheet" href="/css/style.css">  
</head>  
<body>  
  <!-- content -->  
  <!-- JS before </body> -->  
  <script src="/js/main.js"></script>  
</body>  
</html>
```

i Aucune modification Nginx nécessaire

La directive `try_files $uri $uri/ =404` dans la config Nginx gère automatiquement tous les fichiers statiques. Nginx cherche `/var/www/elmzn.be/css/style.css` quand le navigateur demande `/css/style.css` — aucune configuration supplémentaire.

3.3 Copier les fichiers depuis sa machine locale

Depuis ta machine de développement (pas depuis le LXC) :

```
# Copy a single file
scp index.html root@192.168.1.112:/var/www/elmzn.be/

# Copy an entire folder
scp -r ./dist/* root@192.168.1.112:/var/www/elmzn.be/

# Fix permissions after upload
ssh root@192.168.1.112 'chown -R www-data:www-data /var/www/elmzn.be'
```

4. Déployer un site React (build statique)

React se compile en fichiers statiques via `npm run build`. Nginx les sert directement comme un site HTML classique, avec une adaptation pour le routing React.

4.1 Builder le projet React en local

```
# In your React project folder (on your dev machine)
npm install
npm run build

# The build output is in ./build/ (Create React App)
# or ./dist/ (Vite)
```

4.2 Uploader le build sur le LXC

```
# Upload build folder to LXC
scp -r ./build/* root@192.168.1.112:/var/www/elmzn.be/

# Fix permissions
ssh root@192.168.1.112 'chown -R www-data:www-data /var/www/elmzn.be'
```

4.3 Adapter la config Nginx pour React Router

Sans cette modification, les routes React (ex: /about) retournent une erreur 404 au rechargement de la page.

```
server {  
    listen 80;  
    server_name elmzn.be www.elmzn.be;  
    root /var/www/elmzn.be;  
    index index.html;  
  
    location / {  
        # Redirect all routes to index.html  
        # React Router handles client-side routing  
        try_files $uri $uri/ /index.html;  
    }  
  
    access_log /var/log/nginx/elmzn.be.access.log;  
    error_log /var/log/nginx/elmzn.be.error.log;  
}  
  
# Apply changes  
nginx -t && systemctl reload nginx
```

5. Déployer un site Next.js (SSR)

Next.js en mode SSR (Server-Side Rendering) nécessite un serveur Node.js actif. Nginx joue le rôle de reverse proxy vers ce serveur.

⚠ RAM — Vérifier avant de commencer

Le build Next.js peut consommer jusqu'à 1 GB de RAM. Avec 512 MB par défaut, le build risque d'être tué par le kernel (OOM kill). Augmenter si nécessaire depuis le host Proxmox.

```
# Depuis le host Proxmox – augmenter la RAM à 1.5 GB  
pct set 102 --memory 1536  
pct reboot 102
```

5.1 Installer Node.js dans le LXC

```
# Enter LXC
pct enter 102

# Install Node.js 20 LTS
curl -fsSL https://deb.nodesource.com/setup_20.x | bash -
apt install -y nodejs

# Verify
node --version # v20.x.x
npm --version
```

5.2 Uploader le projet (sans node_modules)

Ne jamais copier `node_modules` via `scp` — des centaines de MB inutiles. Utiliser `rsync` avec exclusion, depuis ta machine de développement :

```
# Upload source only – exclude node_modules and build artifacts
rsync -avz --exclude='node_modules' --exclude='.next' \
  ./my-nextjs-app/ root@192.168.1.112:/var/www/app.elmzn.be/

# Install dependencies on the server
ssh root@192.168.1.112 'cd /var/www/app.elmzn.be && npm install'
```

5.3 Variables d'environnement

Les variables d'environnement de production ne doivent jamais être committées dans le dépôt Git. Les créer directement sur le serveur :

```
# Create the production env file on the server
nano /var/www/app.elmzn.be/.env.production

# Example content :
# Public variables (exposed to browser AND server)
NEXT_PUBLIC_API_URL=https://api.elmzn.be

# Private variables (server-side only)
DATABASE_URL=postgres://user:pass@localhost:5432/mydb
SECRET_KEY=your-secret-key
```

i Variables d'environnement Next.js

- `NEXT_PUBLIC_*` → inclus dans le bundle JS — visible par le navigateur.
- Sans préfixe → serveur uniquement (API routes, `getServerSideProps`, Server Components).
- Ne jamais mettre de secrets (tokens, passwords) dans `NEXT_PUBLIC_*`.

5.4 Activer le mode standalone (recommandé)

Le mode standalone réduit la taille du déploiement d'environ 70% et simplifie le démarrage en production. Modifier `next.config.js` sur ta machine de développement :

```
// next.config.js
/** @type {import('next').NextConfig} */
const nextConfig = {
  output: 'standalone',
}

module.exports = nextConfig
```

i Pourquoi standalone ?

En mode standard, `next start` nécessite tous les `node_modules` (~300 MB sur disque). En mode standalone, Next.js ne copie que les dépendances strictement nécessaires dans `.next/standalone` — démarrage plus rapide, empreinte mémoire et disque réduits.

5.5 Builder le projet Next.js

Sur le serveur LXC :

```
cd /var/www/app.elmzn.be
npm run build

# Verify build output
ls .next/
# Expected: cache/  server/  standalone/  static/

# Copy public assets into standalone output (required step)
cp -r public .next/standalone/
cp -r .next/static .next/standalone/.next/
```

5.6 Installer et démarrer avec PM2

PM2 maintient le serveur Node.js actif en permanence et le redémarre automatiquement en cas de crash ou de reboot.

```
# Install PM2 globally
npm install -g pm2

# Start in standalone mode (recommended – see 5.4)
PORT=3001 pm2 start .next/standalone/server.js --name 'app-elmzn'

# Alternative: standard mode (without standalone)
# pm2 start npm --name 'app-elmzn' -- start -- -p 3001

# Save PM2 config to survive reboots
pm2 startup
pm2 save

# Verify
pm2 list
# Expected: app-elmzn  online  ...

# Useful PM2 commands
pm2 logs app-elmzn      # View live logs
pm2 restart app-elmzn   # Restart app
pm2 stop app-elmzn      # Stop app
```

5.7 Configurer Nginx en reverse proxy

Créer un nouveau virtual host pour le sous-domaine :

```
nano /etc/nginx/sites-available/app.elmzn.be
```

```
server {
  listen 80;
  server_name app.elmzn.be;

  location / {
    proxy_pass http://localhost:3001;
    proxy_http_version 1.1;
    proxy_set_header Upgrade $http_upgrade;
    proxy_set_header Connection 'upgrade';
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_cache_bypass $http_upgrade;
  }

  access_log /var/log/nginx/app.elmzn.be.access.log;
  error_log /var/log/nginx/app.elmzn.be.error.log;
}
```

```
# Enable site
ln -s /etc/nginx/sites-available/app.elmzn.be /etc/nginx/sites-enabled/
nginx -t && systemctl reload nginx
```

Type de site	Serveur Node.js	Config Nginx	Complexité
HTML/CSS/JS pur	Non	try_files \$uri =404	Simple
React (build)	Non	try_files \$uri /index.html	Simple
Next.js static export	Non	try_files \$uri /index.html	Simple
Next.js SSR	Oui — PM2	proxy_pass localhost:3001	Intermédiaire

6. Installation et configuration de NPM

Nginx Proxy Manager (NPM) est installé sur la VM-EXTRANET. Il reçoit le trafic Internet et le redirige vers le bon service en interne.

6.1 Installer NPM avec Docker

Se connecter en SSH sur VM-EXTRANET (192.168.1.111), puis :

```
# Create directories
mkdir -p /opt/extranet/npm/data
mkdir -p /opt/extranet/npm/letsencrypt

# Create docker-compose file
nano /opt/extranet/docker-compose.yml

services:
  npm:
    image: jc21/nginx-proxy-manager:latest
    container_name: npm
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
      - "81:81"
    volumes:
      - /opt/extranet/npm/data:/data
      - /opt/extranet/npm/letsencrypt:/etc/letsencrypt

# Start NPM
docker compose -f /opt/extranet/docker-compose.yml up -d

# Verify
docker ps
```

6.2 Accéder au dashboard NPM

Élément	Valeur
URL dashboard	http://192.168.1.111:81
Email par défaut	admin@example.com
Mot de passe par défaut	changeme
Action à la première connexion	Changer email et mot de passe

6.3 Créer un Proxy Host

Dans le dashboard NPM, cliquer sur **Proxy Hosts > Add Proxy Host**.

Onglet Details :

- Domain Names : elmzn.be et www.elmzn.be
- Scheme : http
- Forward Hostname/IP : 192.168.1.112 (IP du LXC)
- Forward Port : 80
- Cocher : Block Common Exploits + Websockets Support

Onglet SSL :

- SSL Certificate : Request a new SSL certificate
- Cocher : Force SSL + HTTP/2 Support
- Email : ton adresse email
- Cocher : I agree to the Let's Encrypt Terms

⚠ Prérequis pour le certificat SSL Let's Encrypt

1. Le DNS du domaine doit pointer vers ton IP publique.
2. Les ports 80 et 443 doivent être accessibles depuis Internet.
3. Si les ports sont bloqués par l'opérateur, utiliser Cloudflare Tunnel.
Voir Section 7 pour la configuration Cloudflare Tunnel.

7. Configuration DNS (OVH)

Le DNS fait le lien entre ton nom de domaine et ton IP publique. Ces étapes se font dans l'interface OVH.

7.1 Récupérer ton IP publique

```
# From Proxmox host or any machine on your network
curl -s https://api.ipify.org
```

```
# Example output: 109.130.209.28
```

7.2 Créer les enregistrements A dans OVH

Dans le manager OVH : **Domain names** > [elmzn.be](#) > **DNS Zone**

Sous-domaine	Type	Cible	TTL
@ (racine)	A	109.130.209.28 (ton IP publique)	Par défaut
www	A	109.130.209.28 (ton IP publique)	Par défaut
blog	A	109.130.209.28 (ton IP publique)	Par défaut
app	A	109.130.209.28 (ton IP publique)	Par défaut

i Propagation DNS

Après modification, la propagation DNS prend entre 5 minutes et 24 heures.

Pour vérifier : `dig elmzn.be +short`

Quand ça retourne ton IP publique, la propagation est effective.

7.3 Ajouter un nouveau sous-domaine (workflow)

1. Créer l'enregistrement A dans OVH (ex: `blog` → IP publique)
2. Créer le dossier du site dans le LXC : `mkdir -p /var/www/blog.elmzn.be`
3. Créer le virtual host Nginx : `/etc/nginx/sites-available/blog.elmzn.be`
4. Activer le site : `ln -s ... && nginx -t && systemctl reload nginx`
5. Créer le Proxy Host dans NPM avec le sous-domaine `blog.elmzn.be`

6. Attendre la propagation DNS puis tester

8. Cloudflare Tunnel (si ports bloqués)

Si l'opérateur Internet bloque les ports 80/443 entrants (cas fréquent chez Proximus en résidentiel), Cloudflare Tunnel est la solution recommandée. Elle contourne complètement le problème.

8.1 Fonctionnement

Sans Cloudflare Tunnel (ports ouverts) :

Visiteur → Internet → Box (port 80/443) → VM-EXTRANET → LXC

Avec Cloudflare Tunnel (ports bloqués) :

Visiteur → Cloudflare → Tunnel chiffré sortant → VM-EXTRANET → LXC

(la VM initie la connexion, pas de port entrant)

i Avantages de Cloudflare Tunnel

- Aucun port forwarding nécessaire sur la box.
- HTTPS automatique géré par Cloudflare (pas besoin de Let's Encrypt).
- Protection DDoS incluse gratuitement.
- Fonctionne même avec une IP publique dynamique.

8.2 Prérequis

- Compte Cloudflare gratuit sur cloudflare.com
- Transférer la gestion DNS de OVH vers Cloudflare (changer les nameservers OVH)

8.3 Installation de cloudflared

Depuis la VM-EXTRANET (192.168.1.111) :

```
# Add Cloudflare GPG key and repository
curl -fsSL https://pkg.cloudflare.com/cloudflare-main.gpg | gpg --dearmor -o /usr/share/keyrings:
echo 'deb [signed-by=/usr/share/keyrings/cloudflare-main.gpg] https://pkg.cloudflare.com/cloudf:

# Install cloudflared
apt update && apt install -y cloudflared
```

8.4 Authentifier et créer le tunnel

```
# Authenticate (opens browser for Cloudflare login)
cloudflared tunnel login

# Create tunnel
cloudflared tunnel create elmzn-tunnel

# Note the tunnel ID returned – you'll need it
# Example: abc123de-f456-7890-abcd-ef1234567890
```

8.5 Configurer le tunnel

```
mkdir -p /etc/cloudflared
nano /etc/cloudflared/config.yml

tunnel: <TUNNEL_ID>
credentials-file: /root/.cloudflared/<TUNNEL_ID>.json

ingress:
  - hostname: elmzn.be
    service: http://192.168.1.112:80
  - hostname: www.elmzn.be
    service: http://192.168.1.112:80
  - hostname: blog.elmzn.be
    service: http://192.168.1.112:80
  - service: http_status:404
```

8.6 Démarrer le tunnel en service systemd

```
# Install as system service
cloudflared service install

# Start and enable
systemctl start cloudflared
systemctl enable cloudflared

# Check status
systemctl status cloudflared
```

9. Gestion multi-sites

Le LXC est conçu pour héberger plusieurs sites simultanément. Voici le workflow complet pour ajouter un nouveau site.

9.1 Structure finale attendue

```
# LXC filesystem
/var/www/
├─ elmzn.be/           ← landing page (HTML pur)
├─ blog.elmzn.be/     ← blog (React static)
└─ app.elmzn.be/       ← application (Next.js SSR)

# Nginx config
/etc/nginx/sites-available/
├─ elmzn.be
├─ blog.elmzn.be
└─ app.elmzn.be

/etc/nginx/sites-enabled/
├─ elmzn.be -> ../sites-available/elmzn.be
├─ blog.elmzn.be -> ../sites-available/blog.elmzn.be
└─ app.elmzn.be -> ../sites-available/app.elmzn.be

# PM2 (Next.js only)
pm2 list
# app-elmzn  online  port 3001
# blog-elmzn online  port 3002
```

9.2 Checklist pour ajouter un nouveau site

- ☐ Ajouter l'enregistrement A dans OVH (sous-domaine → IP publique)
- ☐ Créer le dossier : `mkdir -p /var/www/nouveau-site.elmzn.be`
- ☐ Créer la config Nginx : `/etc/nginx/sites-available/nouveau-site.elmzn.be`
- ☐ Activer le site : `ln -s` et `systemctl reload nginx`
- ☐ Uploader les fichiers du site
- ☐ Ajouter le Proxy Host dans NPM
- ☐ Vérifier la propagation DNS : `dig nouveau-site.elmzn.be +short`
- ☐ Tester l'accès HTTPS dans le navigateur

10. Référence rapide — Commandes utiles

10.1 Proxmox host

```
# LXC management
pct list                # List all LXC containers
pct enter 102           # Enter LXC-WEB
pct start 102           # Start LXC
pct stop 102            # Stop LXC
pct set 102 --onboot 1   # Enable autostart

# VM management
qm list                 # List all VMs
qm start 100            # Start VM-EXTRANET
qm set 100 --onboot 1   # Enable VM autostart
```

10.2 LXC-WEB (Nginx)

```
# Nginx management
nginx -t                # Test configuration syntax
systemctl reload nginx  # Reload config (no downtime)
systemctl restart nginx # Full restart
systemctl status nginx  # Check status

# Logs
tail -f /var/log/nginx/elmzn.be.access.log
tail -f /var/log/nginx/elmzn.be.error.log

# Virtual hosts
ls /etc/nginx/sites-enabled/ # List active sites
ls /etc/nginx/sites-available/ # List all sites
```

10.3 NPM (VM-EXTRANET)

```
# NPM Docker management
docker compose -f /opt/extranet/docker-compose.yml ps
docker compose -f /opt/extranet/docker-compose.yml restart npm
docker logs npm --tail 50

# Dashboard
# http://192.168.1.111:81
```

10.4 Diagnostic réseau

```
# Check your public IP
curl -s https://api.ipify.org

# Check DNS propagation
dig elmzn.be +short

# Test port from outside (use mobile data, not WiFi)
# https://www.yougetsignal.com/tools/open-ports/

# Test Nginx response with custom Host header
curl -H "Host: elmzn.be" http://192.168.1.112

# Check LetsEncrypt logs
cat /opt/extranet/npm/data/logs/letsencrypt.log
```

11. Troubleshooting

Problème : Page par défaut Nginx s'affiche

```
# Check enabled sites
ls /etc/nginx/sites-enabled/

# Remove default if present
rm /etc/nginx/sites-enabled/default
systemctl reload nginx
```

Problème : Erreur 404 sur les routes React

Ajouter `/index.html` dans le `try_files` de la config Nginx :

```
location / {
    try_files $uri $uri/ /index.html; # <-- /index.html manquant
}
```

Problème : Let's Encrypt échoue (timeout)

- Vérifier le port forwarding sur la box (80 et 443 → VM-EXTRANET)
- Vérifier que le DNS pointe vers l'IP publique : `dig elmzn.be +short`
- Tester depuis un réseau externe (4G/5G) sur yougetsignal.com
- Si l'opérateur bloque les ports → utiliser Cloudflare Tunnel (Section 8)

Problème : Next.js inaccessible après reboot

```
# Check PM2 status
```

```
pm2 list
```

```
# If not running, restart
```

```
pm2 restart app-elmzn
```

```
# Ensure PM2 survives reboots
```

```
pm2 startup
```

```
pm2 save
```